Insertion and deletion of an element from Binary Search Tree

```
#include <iostream>

using namespace std;

// Node structure

struct Node {

	int data;

	Node* left;

	Node* right;

Node(int value) {

		data = value;

		left = right = NULL;

	}

};

// Insert function

Node* insert(Node* root, int value) {

	if (root == NULL) {

		return new Node(value);

	}



	if (value < root->data) {

		root->left = insert(root->left, value);

	} else if (value > root->data) {

		root->right = insert(root->right, value);

	}

return root;
```

```
}

// Find minimum value node

Node* findMin(Node* root) {

	while (root->left != NULL) {

		root = root->left;

	}

	return root;

}

// Delete function

Node* deleteNode(Node* root, int key) {

	if (root == NULL) return root;



	if (key < root->data) {

		root->left = deleteNode(root->left, key);

	}

	else if (key > root->data) {

		root->right = deleteNode(root->right, key);

	}

	else {

		// Case 1: No child

		if (root->left == NULL && root->right == NULL) {

			delete root;

			return NULL;

		}

		// Case 2: One child
```

```
            else if (root->left == NULL) {
                Node* temp = root->right;
                delete root;
                return temp;
            }
            else if (root->right == NULL) {
                Node* temp = root->left;
                delete root;
                return temp;
            }
            // Case 3: Two children
            Node* temp = findMin(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data);
        }
        return root;
}
// Inorder traversal
void inorder(Node* root) {
        if (root == NULL) return;
        inorder(root->left);
        cout << root->data << " ";
        inorder(root->right);
}
// Main function
```

```
int main() {
    Node* root = NULL;
// Insert elements
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);
cout << "Inorder before deletion: ";
    inorder(root);
    cout << endl;
// Delete element
    root = deleteNode(root, 50);
cout << "Inorder after deletion: ";
    inorder(root);
    cout << endl;
return 0;
}
```

Output

```
Inorder before deletion: 20 30 40 50 60 70 80
Inorder after deletion: 20 30 40 60 70 80


=== Code Execution Successful ===
```